CHAPTER 1

KEY-VALUE STORE

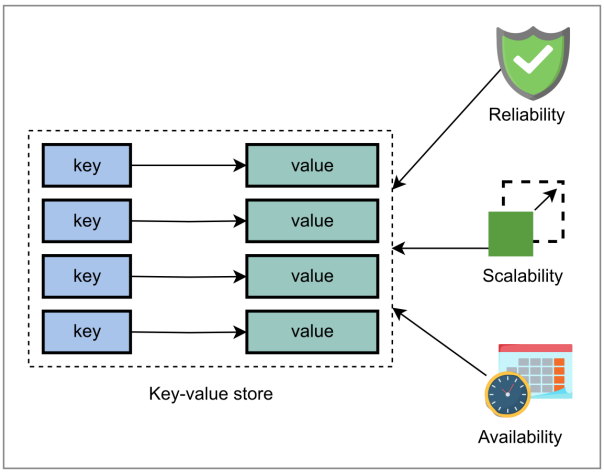
1 System Design: The Key-Value Store 2
2 Design of a Key-Value Store 3
3 Ensure Scalability and Replication . 5
4 Versioning Data and Achieving Configurability 14
5 Enable Fault Tolerance and Failure Detection 20

Learn how to design a scalable key-value store, focusing on replication, versioning, and fault tolerance.

1 System Design: The Key-Value Store

1.1 Introduction to key-value stores

Key-value stores are distributed hash tables (DHTs)(A distributed hash table (DHT) is a decentralized storage system that provides lookup and storage schemes similar to a hash table, storing key-value pairs. Source: https://www.educative.io/edpresso/what-is-a-distributed-hash-table). A key is generated by the hash function and should be unique. In a key-value store, a key binds to a specific value and doesn't assume anything about the structure of the value. A value can be a blob, image, server name, or anything the user wants to store against a unique key.
Usually, it's preferred to keep the size of value relatively smaller (KB to MB). We can put large data in the blob store and put links to that data in the value field. Key -value stores are useful in many situations, such as storing user sessions in a web application and building NoSQL databases.
It's challenging to scale traditional databases with strong consistency and high availability in a distributed environment. Many real-world services like Amazon, Facebook, Instagram, Netflix, and many more use primary-key access to a data store instead of traditional online transaction processing (OLTP) databases. (Online transaction processing (OLTP) involves gathering input information, processing the data, and updating existing data to reflect the collected and processed information. The OLTP databases carry out day-to-day tasks like insertion, updating, and deletion of the data in the database.)Examples of key-value store usage include bestseller lists, shopping carts, customer preferences, session management, sales rank, and product catalogs.



> **Note:** Many applications might not require a rich programming model provided by a traditional relational database management system (RDBMS). Using RDBMS for such applications is often expensive in terms of cost and performance.

1.2 How will we design a key-value store?

We've divided the key-value system design into the following four lessons:

1. **Design a Key-value Store**: We'll define the requirements of a key-value store and design the API.
2. **Ensure Scalability and Replication**: We'll learn to achieve scalability using consistent hashing and replicate the partitioned data.
3. **Versioning Data and Achieving Configurability**: We'll learn to resolve conflicts that occur due to changes made by more than one entity, and we'll make our system more configurable for different use cases.

4. **Enable Fault Tolerance and Failure Detection**: We'll learn to make a key-value store fault-tolerant and how to detect failures in the system.

2 Design of a Key-Value Store

2.1 Requirements

Let's list the requirements of designing a key-value store to overcome the problems of traditional databases.

Functional requirements

Typically, key-value stores are expected to offer functions such as `get` and `put`. However, what sets this particular key-value store system apart is its distinct characteristics, explained as follows:

- **Configurable service**: Some applications might have a tendency to trade strong consistency for higher availability. We need to provide a configurable service so that different applications could use a range of consistency models. We need tight control over the trade-offs between availability, consistency, cost-effectiveness, and performance.

  Note: Such configurations can only be performed when instantiating a new key-value store instance and cannot be changed dynamically when the system is operational.

- **Ability to always write (when we picked "A" over "C" in the context of CAP)**: The applications should always have the ability to write into the key-value storage. If the user wants strong consistency, this requirement might not always be fulfilled due to the implications of the CAP theorem.

  Note: The context of the problem determines what will be classified as a functional requirement and what will be classified as non-functional. For example, the ability to always write (high availability) is a functional requirement for Amazon's shopping cart application, while in other cases, high availability may be considered a non-functional requirement. Drawing inspiration from Amazon's Dynamo key-value store, we can categorize the ability to always write as a functional requirement.

- **Hardware heterogeneity**: We want to add new servers with different and higher capacities, seamlessly, to our cluster without changing or upgrading existing servers. Our system should be able to accommodate and leverage different capacity servers, ensuring correct core functionality (`get` and `put` data) while balancing the workload distribution according to each server's capacity. This calls for a peer-to-peer design with no distinguished nodes.

Non-functional requirements

The non-functional requirements are as follows:

- **Scalability**: Key-value stores should run on tens of thousands of servers distributed across the globe. Incremental scalability is highly desirable. We should add or remove the servers as needed with minimal to no disruption to the service availability. Moreover, our system should be able to handle an enormous number of users of the key-value store.

- **Fault tolerance**: The key-value store should operate uninterrupted despite failures in servers or their components.

Now that you understand the functional and non-functional requirements of a key-value store, what do you think are the key differences between key-value stores and traditional databases?
In what scenarios are key-value stores particularly advantageous?

Question:

Differences between key-value stores and traditional databases

Reference Answer:

Differences: Key-value stores focus on simple retrieval by key, don't have complex query languages, and trade strict consistency for availability and scalability in many cases. Key -value stores have unstructured data. Advantages : Key-value stores excel at handling massive amounts of data, high-speed data lookups, and situations where the data structure may be flexible or not fully known in advance.

Running a key-value store on a single server limits scalability and reliability. No single machine can handle large-scale data and query demands.
If that server fails, the entire system goes down. Distributing data across multiple servers helps scale storage and ensures continuous availability.

2.2 Assumptions

We'll assume the following to keep our design simple:

- The data centers hosting the service are trusted (non-hostile).
- All the required authentication and authorization are already completed.
- User requests and responses are relayed over HTTPS.

2.3 API design

Key-value stores, like ordinary hash tables, provide two primary functions, which are `get` and `put`.
Let's look at the API design.

The `get` function

The API call to get a value should look like this:

```
get(key)
```

We return the associated value on the basis of the parameter `key`. When data is replicated, it locates the object replica associated with a specific key that's hidden from the end user. It 's done by the system if the store is configured with a weaker data consistency model. For example, in eventual consistency, there might be more than one value returned against a key.

Parameter	Description
`key`	It's the `key` against which we want to get `value`.

The `put` function

The API call to put the value into the system should look like this:

```
put(key, value)
```

It stores the `value` associated with the `key`. The system automatically determines where data should be placed. Additionally , the system often keeps metadata about the stored object. Such metadata can include the version of the object.

Parameter	Description
`key`	It's the `key` against which we have to store `value`.
`value`	It's the object to be stored against the `key`.

When performing data integrity checks, we often store hashes of values (and sometimes values along with their associated key) as metadata. Should these hashes be generated before or after data compression or encryption, and why?

Question:

Best practice for data integrity checks

Reference Answer:

In most data integrity use cases, hashes should be generated after compression but before encryption.

- **Hashing before encryption** is recommended because encryption produces ciphertext, which may vary across operations due to salting or IVs, even if the underlying data is the same. Hashing encrypted data would make integrity checks unreliable.
- **Hashing after compression** is preferred because compression reduces data size without changing meaning, and it avoids hashing redundant data unnecessarily. So, the typical and most effective sequence is: Compress → Hash → Encrypt. This ensures integrity is validated on the actual content, not the encrypted form, and it supports consistent verification during both storage and retrieval operations. **Example scenario:** Suppose you store daily database backups in a cloud archive. First, you compress each backup to reduce its size. Then, you generate a hash of the compressed file and store it as metadata. Finally, you encrypt the file before uploading it. Later, when retrieving the backup, you decrypt it and re-hash the compressed file to compare with the stored hash. This way, you verify that the actual content is intact, regardless of encryption changes or storage format.

Data type

The key is often a primary key in a key-value store, while the value can be any arbitrary binary data.

> **Note:** Dynamo uses MD5 hashes on the key to generate a 128-bit identifier. These identifiers help the system determine which server node will be responsible for this specific key.

In the next lesson, we'll learn how to design our key-value store. First , we'll focus on adding scalability, replication, and versioning of our data to our system. Then , we'll ensure the functional requirements and make our system fault-tolerant. We 'll fulfill a few of our non-functional requirements first because implementing our functional requirements depends on the method chosen for scalability.

> **Note:** This chapter is based on Dynamo(Amazon's Highly Available Key-value), which is an influential work in the domain of key-value stores.

3 Ensure Scalability and Replication

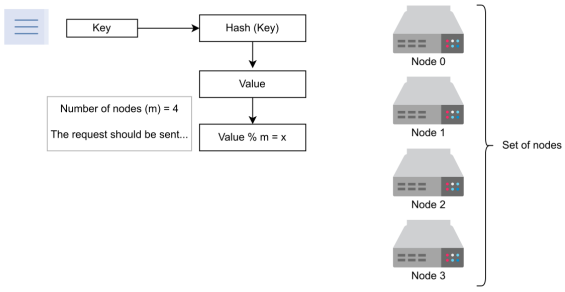
3.1 Add scalability

Let's start with one of the core design requirements: scalability. We store key-value data in storage nodes. With a change in demand, we might need to add or remove storage nodes. It means we need to partition data over the nodes in the system to distribute the load across all nodes.

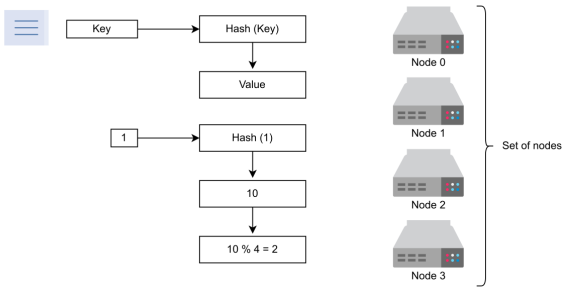
For example, let's consider that we have four nodes, and we want 25% of the requests to go to each node to balance the load equally. The traditional way to solve this is through the modulus operator. Each request that arrives has a key associated with it. When a request comes in, we calculate the hash of the key. Then , we find the remainder by taking the modulus of the hashed value with the number of nodes `m`. The remainder value `x` is the node number, and we send the request to that node to process it.

The following slides explain this process:

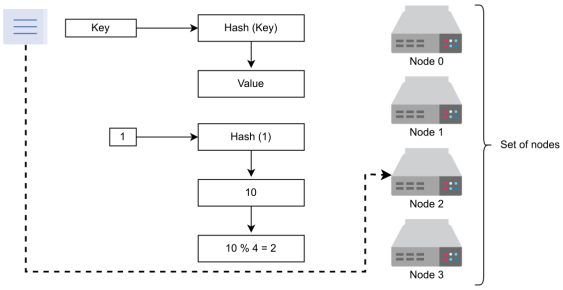
We want to add and remove nodes with minimal change in our infrastructure. But in this method, when we add or remove a node, we need to move a lot of keys. This is inefficient. For example, node 2 is removed, and suppose for the same key, the new server to process a request will be node 1 because $10\%3 = 1$. Nodes



(a) We get the hash of the key and take modulus with the number of nodes to find the node that should process the request



(b) We perform the required operations on the key to get the node



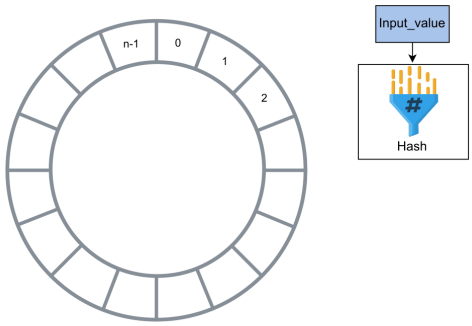
(a) Node 2 will process the request

hold information in their local caches, like keys and their values. So , we need to move that request's data to the next node that has to process the request. But this replication can be costly and can cause high latency. In distributed key-value stores, consistent hashing decides which node stores a given key.
This direct mapping lets requests reach the correct node immediately, maintaining data locality and low latency.
A load balancer would interfere by spreading requests randomly, leading to misrouted traffic, extra cross-node communication, and higher latency. Consistent hashing already handles both load distribution and scalability without adding extra routing layers.
Next, we'll learn how to copy data efficiently.

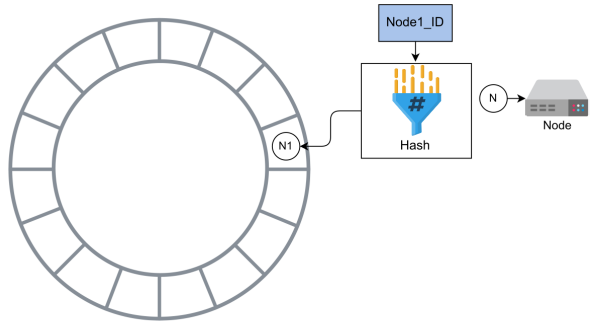
Consistent hashing

Consistent hashing is an effective way to manage the load over the set of nodes. In consistent hashing, we consider that we have a conceptual ring of hashes from 0 to $n-1$, where n is the number of available hash values. We use each node's ID, calculate its hash, and map it to the ring. We apply the same process to requests. Each request is completed by the next node that it finds by moving in the clockwise direction in the ring.
Whenever a new node is added to the ring, the immediate next node is affected. It has to share its data with the newly added node while other nodes are unaffected. It 's easy to scale since we're able to keep changes to our nodes minimal. This is because only a small portion of overall keys need to move. The hashes are randomly distributed, so we expect the load of requests to be random and distributed evenly on average on the ring.

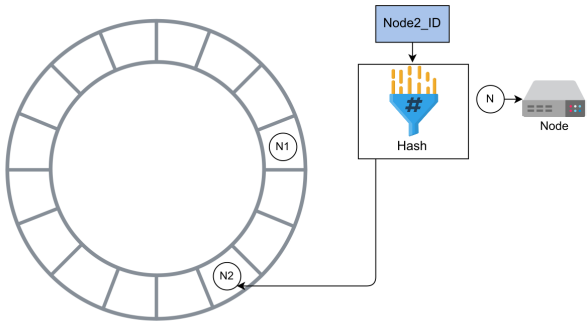
The primary benefit of consistent hashing is that as nodes join or leave, it ensures that a minimal number of keys need to move. However, the request load isn't equally divided in practice. Any server that handles a large chunk of data can become a bottleneck in a distributed system. That node will receive a disproportionately large share of data storage and retrieval requests, reducing the overall system performance. As a result, these are referred to as hotspots.
As shown in the figure below, most of the requests are between the N4 and N1 nodes. Now , N1 has to handle



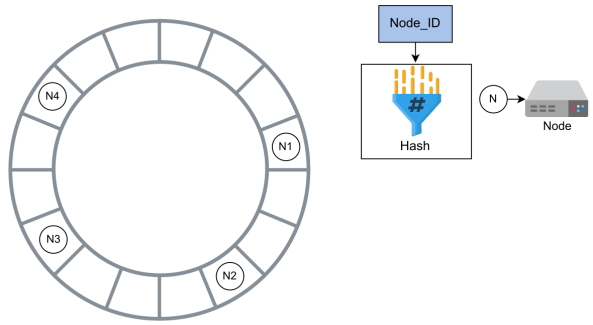
(a) Consider we have a conceptual ring of hashes from 0 to n-1, where n is the total number of hash values in the ring



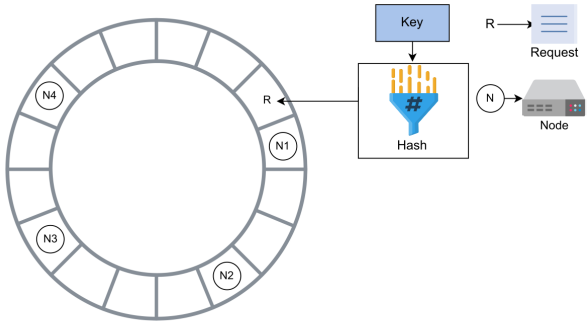
(b) A hash for Node1 is calculated and Node1 is added to the ring



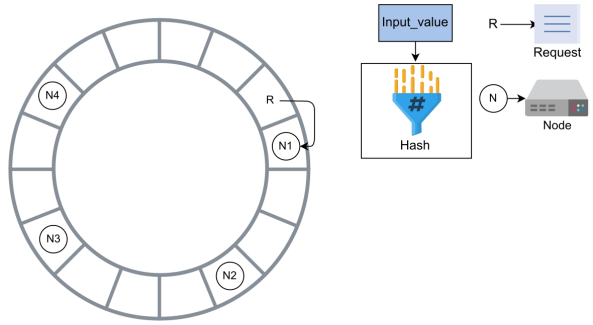
(a) A hash for Node2 is calculated and Node2 is added to the ring



(b) Hashes for other nodes are calculated and nodes are added to the ring



(a) Similarly, a hash is calculated for the key and the request is added to the ring



(b) The request is completed by the next node it finds by moving in the clockwise direction

(a) The hash is calculated for the next request and the request is added to the ring

(b) The request is completed by N2 since it's the next node in the clockwise direction

(a) The hash is calculated for the request and the request is added to the ring

(b) The request is completed by N3 since it's the next node in the clockwise direction

(a) The hash is calculated for a new node, Node5, which is added to the ring

(b) N3 shares the keys from N2 to N5 with N5, since it is the responsibility of N5 it process the requests that comes after N2 and before N5

(a) The hash is calculated for a new request and the request is added to the ring

(b) The request is processed by N5 instead of N3 since N5 is the next node in the clockwise direction

most of the requests compared to other nodes, and it has become a hotspot. That means non-uniform load distribution has increased load on a single server.

> **Note:** It's a good exercise to think of possible solutions to the non-uniform load distribution before reading on.

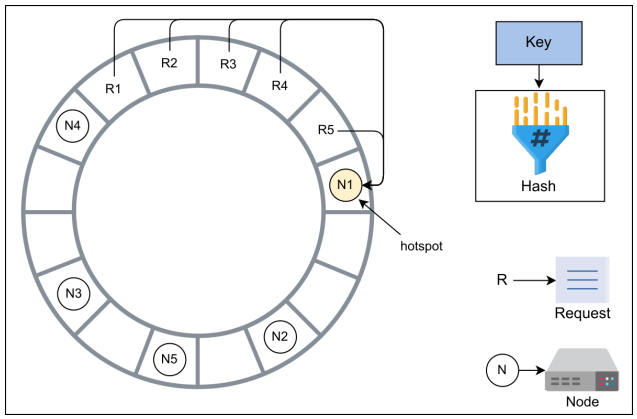
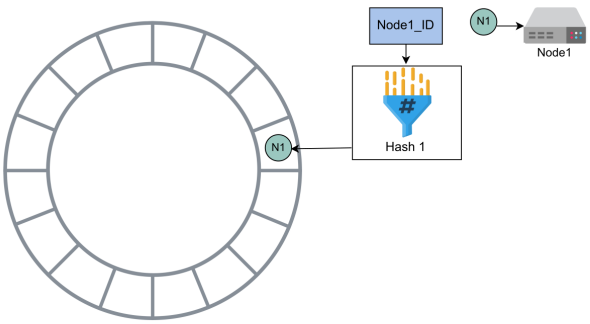
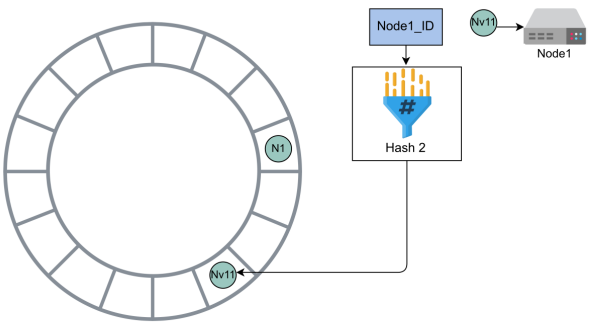


Figure 1.10: Non-uniform request distribution in the ring

Use virtual nodes We'll use virtual nodes to ensure a more evenly distributed load across the nodes. Instead of applying a single hash function, we'll apply multiple hash functions onto the same key.
Let's take an example. Suppose we have three hash functions. For each node, we calculate three hashes and place them into the ring. For the request, we use only one hash function. Wherever the request lands onto the ring, it's processed by the next node found while moving in the clockwise direction. Each server has three positions, so the load of requests is more uniform. Moreover , if a node has more hardware capacity than others, we can add more virtual nodes by using additional hash functions. This way, it'll have more positions in the ring and serve more requests.



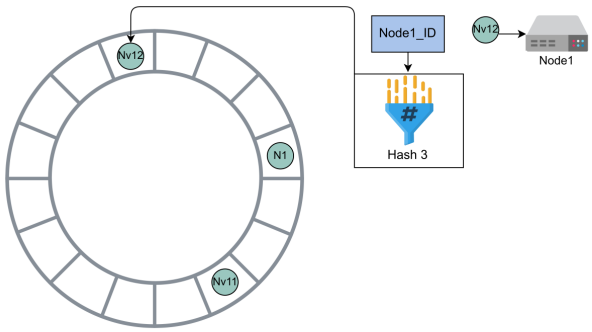
(a) Calculate the hash for Node1 using Hash 1, and place the node in the ring



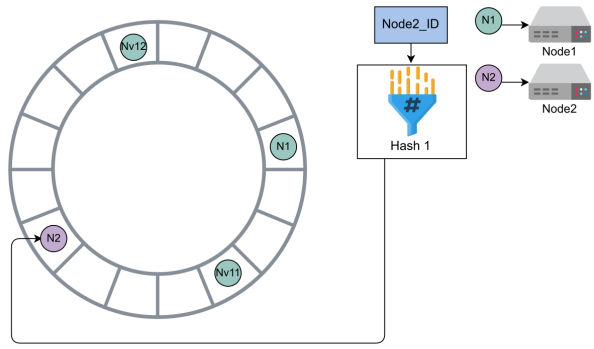
(b) Calculate the hash for the first virtual node of Node1 using Hash 2, and place the node in the ring

Advantages of virtual nodes Following are some advantages of using virtual nodes:

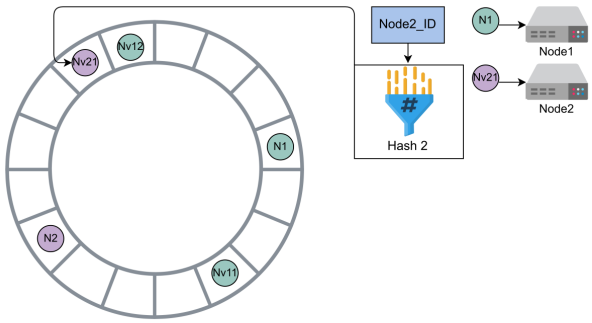
- If a node fails or does routine maintenance, the workload is uniformly distributed over other nodes. For each newly accessible node, the other nodes receive nearly equal load when it comes back online or is added to the system.



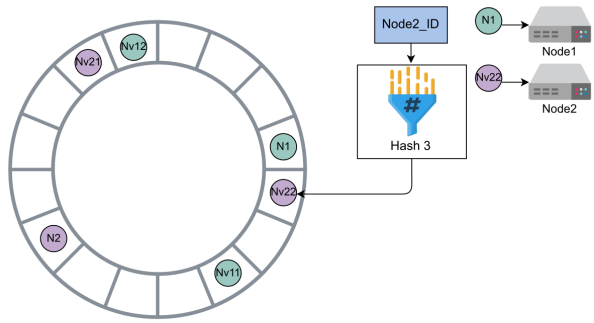
(a) Calculate the hash for the second virtual node of Node1 using Hash 3, and place the node in the ring



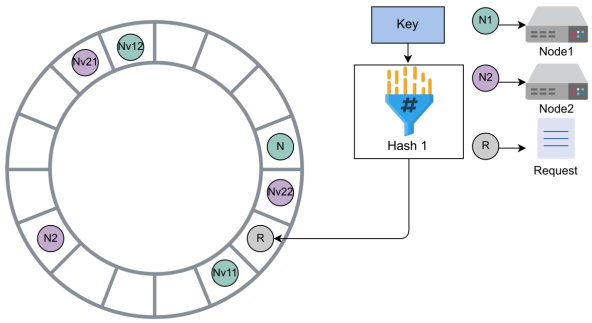
(b) Calculate the hash for Node2 using Hash 1, and place the node in the ring



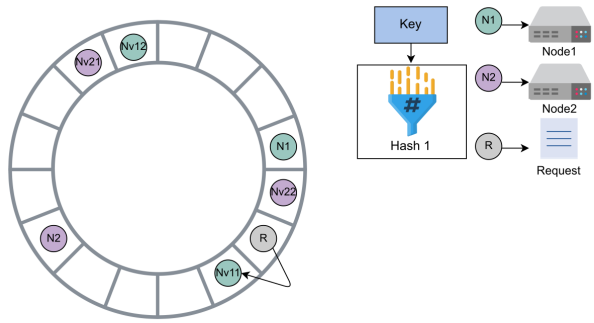
(a) Calculate the hash for the first virtual node of Node2 using Hash 2, and place the node in the ring



(b) Calculate the hash for the second virtual node of Node2 using Hash 3, and place the node in the ring



(a) Calculate the hash for the request using Hash 1, and place the request in the ring



(b) The request will be processed by the first virtual node of Node1 as requests are processed in the clockwise direction

- It's up to each node to decide how many virtual nodes it's responsible for, considering the heterogeneity(The system needs to exploit heterogeneity in its infrastructure. e.g., the work distribution must be proportional to the capabilities of the individual servers. It is essential to add new nodes with higher capacity without upgrading all hosts.) of the physical infrastructure. For example, if a node has roughly double the computational capacity as compared to the others, it can take more load.

We've made the proposed design of key-value storage scalable. The next task is to make our system highly available.
Describe how a key-value store can support incremental scalability without disrupting service availability.
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:
How to achieve incremental scalability?
Reference Answer:
A key-value store can support incremental scalability without disrupting service availability by adopting the following techniques:

- Use consistent hashing to redistribute keys only among affected nodes when adding/removing nodes.
- Employ virtual nodes to minimize data movement during scaling.
- Perform data migration in the background with minimal impact on active operations.
- Implement read/write forwarding to handle in-transit data during scaling transitions.

3.2 Data replication

We have various methods to replicate the storage. It can be either a primary-secondary relationship or a peer-to-peer relationship.

Primary-secondary approach

In a **primary-secondary** approach, one of the storage areas is primary, and other storage areas are secondary. The secondary replicates its data from the primary. The primary serves the write requests while the secondary serves read requests. After writing, there's a lag for replication. Moreover, if the primary goes down, we can't write into the storage, and it becomes a single point of failure.

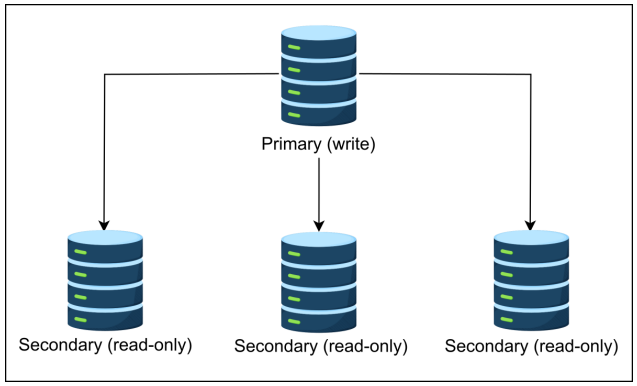


Figure 1.15: Primary-secondary approach

Does the primary-secondary approach fulfill the requirements of the key-value store that we defined in the System Design: The Key-value Store lesson?

Question:
Primary-secondary approach

Reference Answer:

One of our requirements is that we need the ability to always write. This approach is good for the always read option. However, this approach doesn't include the ability to always write because it will overload the primary storage. Moreover , if a primary server fails, we need to upgrade a secondary to a primary. The availability of write will suffer as we won't allow writes during the switch-over time.

Peer-to-peer approach

In the **peer-to-peer** approach, all involved storage areas are primary, and they replicate the data to stay updated. Both read and write are allowed on all nodes. Usually, it's inefficient and costly to replicate in all nnn nodes. Instead, three or five is a common choice for the number of storage nodes to be replicated.

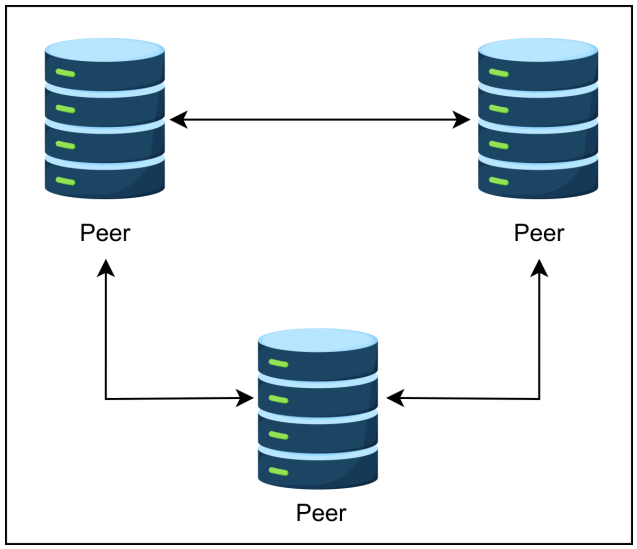


Figure 1.16: Peer-to-peer relationship

We'll use a peer-to-peer relationship for replication. We 'll replicate the data on multiple hosts to achieve durability and high availability. Each data item will be replicated at n hosts, where n is a parameter configured per instance of the key-value store. For example, if we choose n to be 5, it means we want our data to be replicated to five nodes.

Each node will replicate its data to the other nodes. We 'll call a node coordinator that handles read or write operations. It 's directly responsible for the keys. A coordinator node is assigned the key "K." It's also responsible for replicating the keys to $n-1$ successors on the ring (clockwise). These lists of successor virtual nodes are called preference lists. To avoid putting replicas on the same physical nodes, the preference list can skip those virtual nodes whose physical node is already in the list.

Let's consider the illustration given below. We have a replication factor, n, set to 3. For the key "K," the replication is done on the next three nodes: B, C, and D. Similarly , for key "L," the replication is done on nodes C, D, and E.

What is the impact of synchronous or asynchronous replication?

Question:

Synchronous and asynchronous replication

Reference Answer:

In synchronous replication, each write must be acknowledged by all replicas before it is considered complete. This process slows down performance but guarantees strong consistency. However, if some replicas are

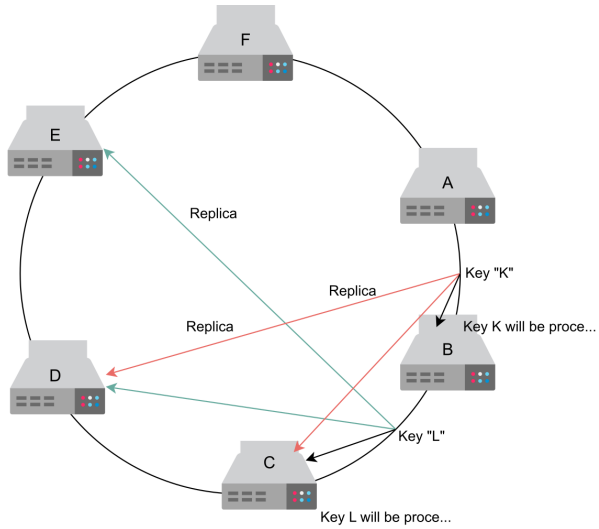


Figure 1.17: Replication in key-value store

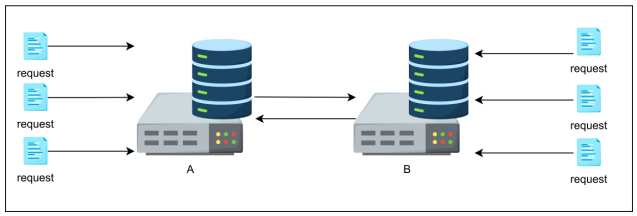
unreachable, availability is reduced. In asynchronous replication, writes finish without waiting for all replicas, which improves performance and availability but can result in temporary inconsistencies. These outcomes reflect the CAP theorem, where systems must balance consistency and availability when network partitions occur.
In the context of the CAP theorem, key-value stores can either be consistent or be available when there are network partitions. For key-value stores, we prefer availability over consistency. It means if the two storage nodes lost connection for replication, they would keep on handling the requests sent to them, and when the connection is restored, they'll sync up. In the disconnected phase, it's highly possible for the nodes to be inconsistent. So, we need to resolve such conflicts. In the next lesson, we'll learn a concept to handle inconsistencies using the versioning of our data.

4 Versioning Data and Achieving Configurability

4.1 Data versioning

When network partitions and node failures occur during an update, an object's version history(The version history contains the details of objects that have been modified.) might become fragmented. As a result, it requires a reconciliation effort on the part of the system. It 's necessary to build a way that explicitly accepts the potential of several copies of the same data so that we can avoid the loss of any updates. It 's critical to realize that some failure scenarios can lead to multiple copies of the same data in the system. So , these copies might be the same or divergent. Resolving the conflicts among these divergent histories is essential and critical for consistency purposes.

To handle inconsistency, we need to maintain causality between the events. We can do this using the timestamps and update all conflicting values with the value of the latest request. But time isn't reliable in a distributed system, so we can't use it as a deciding factor.
Another approach to maintaining causality effectively is by using vector clocks. A **vector clock** is a list of (node, counter) pairs. There's a single vector clock for every version of an object. If two objects have different vector clocks, we're able to tell whether they're causally related or not (more on this in a bit). Unless one of the two changes is reconciled, the two are deemed at odds.



Explain how metadata like versioning and checksums, which detect data corruption, help maintain data integrity and consistency in a key-value store.

Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

How do we ensure data integrity in a key-value store?

Reference Answer:

Metadata like versioning and checksums play a crucial role in maintaining data integrity and consistency in a key-value store. Here 's how they contribute:

- **Versioning:**
  - Tracks changes made to data, enabling conflict resolution during replication (e.g., vector clocks).
  - Helps clients identify the latest version of data in eventually consistent systems.
- **Checksums:**
  - Ensures data integrity by verifying that the data hasn't been tampered with or corrupted during transmission.
  - Taken after compression or encryption to match the stored data format.
- These mechanisms improve reliability and facilitate recovery in distributed environments.

Modify the API design

We talked about how we can decide if two events are causally related or not using a vector clock value. For this, we need information about which node performed the operation before and what its vector clock value was. This is the context of an operation. So, we'll modify our API design as follows.
The API call to get a value should look like this:

```
get(key)
```

Parameter	Description
`key`	This is the `key` against which we want to get `value`.

We return an object or a collection of conflicting objects along with a `context`. The `context` holds encoded metadata about the object, including details such as the object's version.
The API call to put the value into the system should look like this:

```
put(key, context, value)
```

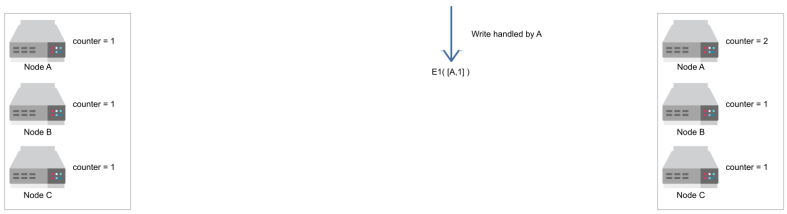
Parameter	Description
`key`	This is the `key` against which we have to store `value`.
`context`	This holds the metadata for each object.
`value`	This is the object that needs to be stored against the `key`.

The function finds the node where the value should be placed on the basis of the `key` and stores the value associated with it. The `context` is returned by the system after the `get` operation. If we have a list of objects in `context` that raises a conflict, we'll ask the client(Here the client is any front-end server in our trusted data center. It does not mean the end-user.) to resolve it.
To update an object in the key-value store, the client must give the `context`. We determine version information using a vector clock by supplying the `context` from a previous read operation. If the key-value store has access to several branches, it provides all objects at the leaf nodes, together with their respective version information in context, when processing a read request. Reconciling disparate versions and merging them into a single new version is considered an update.

> **Note**: This process of resolving conflicts is comparable to how it's done in Git. If Git is able to merge multiple versions into one, merging is performed automatically. It's up to the client (the developer) to resolve conflicts manually if automatic conflict resolution is not possible. Along the same lines, our system can try automatic conflict resolution and, if not possible, ask the application to provide a final resolved value.

Vector clock usage example

Let's consider an example. Say we have a write operation request. Node A handles the first version of the write request, E1; where E means event. The corresponding vector clock has node information and its counter—that is, [A,1]. Node A handles another write for the same object on which the previous write was performed. So , for E2, we have [A,2]. E1 is no longer required because E2 was updated on the same node. E2 reads the changes made by E1, and then new changes are made. Suppose a network partition happens. Now , the request is handled by two different nodes, B and C. The context with updated versions, which are E3, E4, and their related clocks, which are ([A,2], [B,1]) and ([A,2],[C,1]), are now in the system.
Suppose the network partition is repaired, and the client requests a write again, but now we have conflicts. The context ([A,2],[B,1],[C,1]) of the conflicts are returned to the client. After the client does reconciliation and A coordinates the write, we have E5 with the clock ([A,3],[B,1],[C,1]).

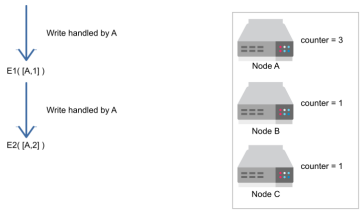


(a) Let's suppose that we have three nodes. The vector clock counter is set to 1 for all of them

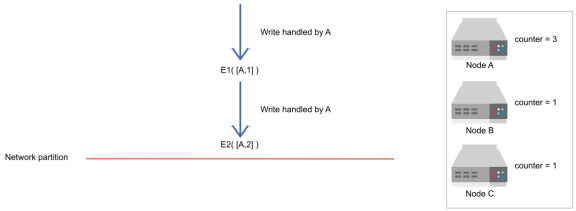
(b) Node A handles the first version of the write request, E1, and the vector clock counter is increased by 1

Compromise with vector clocks limitations

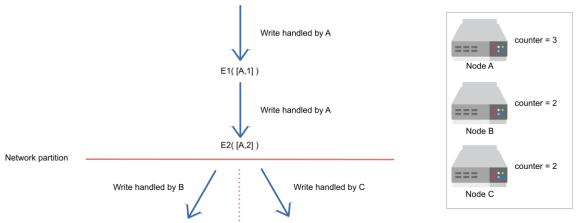
The size of vector clocks may increase if multiple servers write to the same object simultaneously. It 's unlikely to happen in practice because writes are typically handled by one of the top n nodes in a preference list.



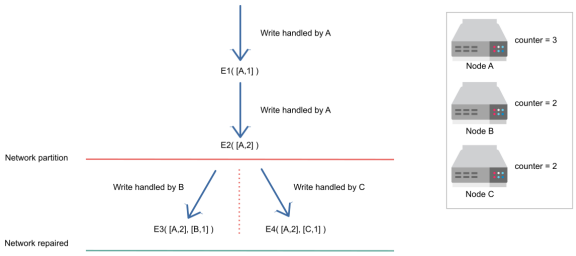
(a) Node A handles the second version of the write request, E2, and the vector clock counter is increased by 2



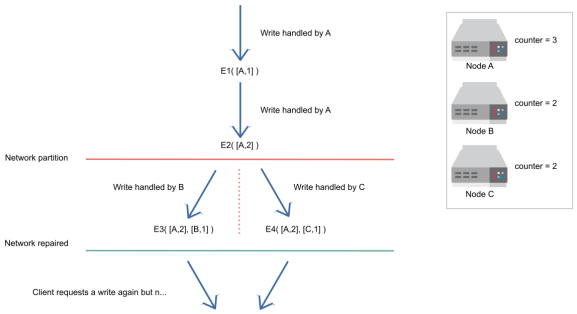
(b) Let's suppose that a network partition happens



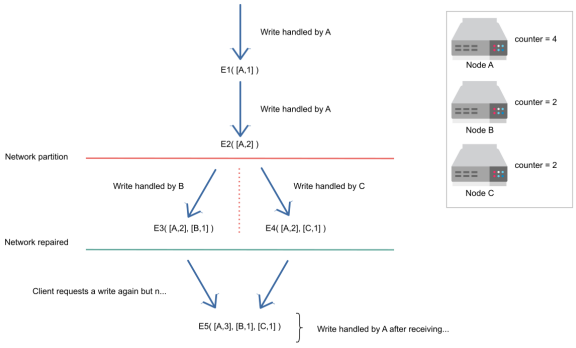
(a) Now, the request is handled by Nodes B and C, and their respective vector clock counter is increased



(b) Let's suppose that the network has now been repaired



(a) The request is sent to Node A to be processed, but now it has conflicts. We ask the client to resolve it



(b) The request is updated after reconciliation

For example, if there are network partitions or multiple server failures, write requests may be processed by nodes not in the top n nodes in the preference list. As a result we can have a long version like this: [A,10],[B,4],[C,1],[D,2],[E,1],[F,3],[G,5],[H,7],[I,2],[J,2],[K,1],[L,1]). It 's a hassle to store and maintain such a long version history.
We can limit the size of the vector clock in these situations. We employ a clock truncation strategy to store a timestamp with each (node, counter) pair to show when the data item was last updated by the node. Vector clock pairs are purged when the number of (node, counter) pairs exceeds a predetermined threshold (let's say 10). Because the descendant linkages can't be precisely calculated, this truncation approach can lead to a lack of efficiency in reconciliation.

The `get` and `put` operations

One of our functional requirements is that the system should be configurable. We want to control the trade-offs between availability, consistency, cost-effectiveness, and performance. So , let's achieve configurability by implementing the basic `get` and `put` functions of the key-value store.
Every node can handle the `get` (read) and `put` (write) operations in our system. A node handling a read or write operation is known as a **coordinator**(A coordinator node is the one where a request first comes in from the client. Client library keeps the information that which request should go to which node.). The coordinator is the first among the top n nodes in the preference list.
There can be two ways for a client to select a node:

- We route the request to a generic load balancer.
- We use a partition-aware client library that routes requests directly to the appropriate coordinator nodes.

Both approaches have their benefits. The client isn't linked to the code in the first approach, whereas lower latency is achievable in the second. The latency is lower due to the reduced number of hops because the client can directly go to a specific server.
Let's make our service configurable by having an ability where we can control the trade-offs between availability, consistency, cost-effectiveness, and performance. We can use a consistency protocol similar to those used in quorum systems(A quorum is the minimum number of votes that a distributed transaction has to obtain in order to be allowed to perform an operation in a distributed system. A quorum-based technique is implemented to enforce consistent operation in a distributed system. (Wikipedia)).
Let's take an example. Say n in the top n of the preference list is equal to 3. This means three copies of the data need to be maintained. We assume that nodes are placed in a ring. Say A, B, C, D, and E is the clockwise order of the nodes in that ring. If the write function is performed on node A, then the copies of that data will be placed on B and C. This is because B and C are the next nodes we find while moving in a clockwise direction of the ring.

4.2 Usage of r and w

Now, consider two variables, r and w. The r means the minimum number of nodes that need to be part of a successful read operation, while w is the minimum number of nodes involved in a successful write operation. So if r = 2, it means our system will read from two nodes when we have data stored in three nodes. We need to pick values of r and w such that at least one node is common between them. This ensures that readers could get the latest-written value. For that, we'll use a quorum-like system by setting r + w > n.

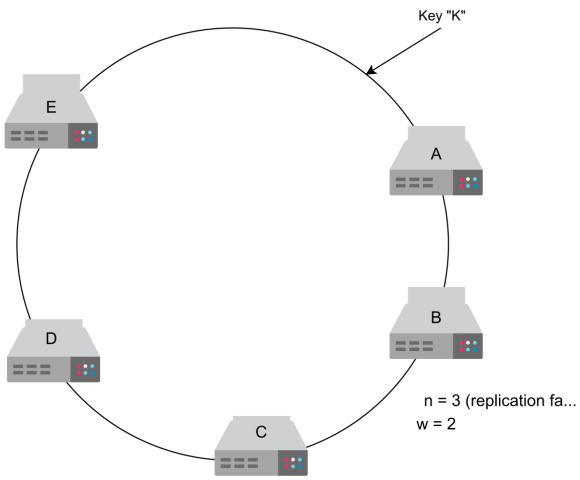
The following table gives an overview of how the values of n, r, and w affect the speed of reads and writes:

Value Effects on Reads and Writes

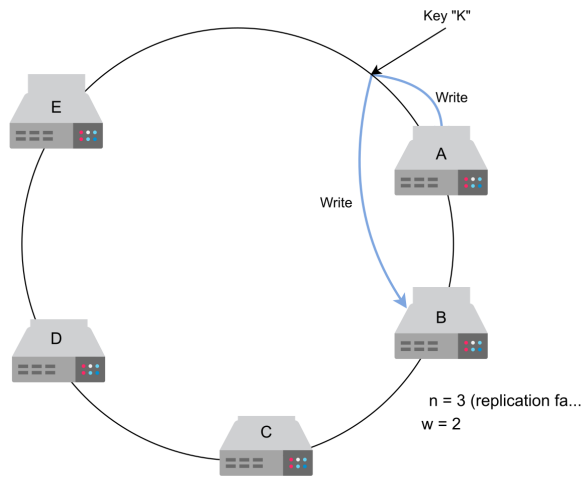
Let's say n = 3, which means we have three nodes where the data is copied to. Now , for w = 2, the operation makes sure to write in two nodes to make this request successful. For the third node, the data is updated asynchronously.

n	r	w	Description
3	2	1	It won't be allowed as it violates our constraint $r + w > n$.
3	2	2	It will be allowed as it fulfills constraints.
3	3	1	It will provide speedy writes and slower reads since readers need to go to all n replicas for a value.
3	1	3	It will provide speedy reads from any node but slow writes since we now need to write to all n nodes synchronously.

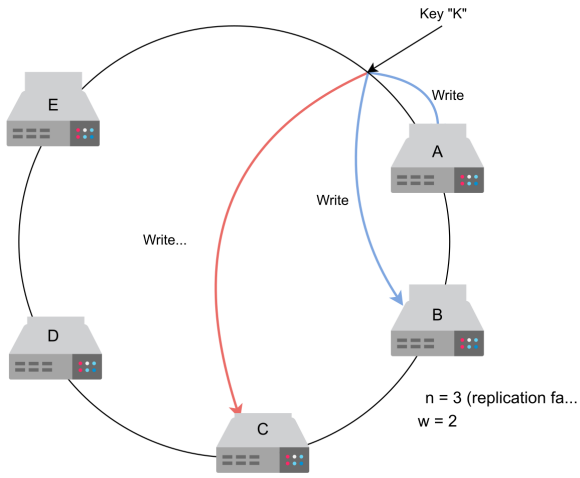
Table 1.1: Value Effects on Reads and Writes



(a) We have a replication factor of 3 and w is 2. The key "K" will be replicated to A, B, and C



(b) Since w=2, we'll write in the first two nodes, then send an acknowledgment to the user or client



(a) For the third node, we'll write/replicate the data asynchronously

In this model, the latency of a get operation is determined by the slowest of the r replicas. As r increases, we require more replicas to complete a read, which can increase latency and decrease availability. At the same time, the system becomes more consistent, as querying more replicas increases the chance of reading the latest value.
The coordinator produces the vector clock for the new version and writes the new version locally upon receiving a `put()` request for a key. The coordinator sends n highest-ranking nodes with the updated version and a new vector clock. We consider a write successful if at least w-1 nodes respond. Remember that the coordinator writes to itself first, so we get w writes in total.
Requests for a `get()` operation are made to the n highest-ranked reachable nodes in a preference list for a key. They wait for r answers before returning the results to the client. Coordinators return all dataset versions that they regard as unrelated if they get several datasets from the same source (divergent histories that need reconciliation). The conflicting versions are then merged, and the resulting key's value is rewritten to override the previous versions.
By now, we've fulfilled the scalability, availability, conflict-resolution, and configurable service requirements. The last requirement is to have a fault-tolerant system. Let's discuss how we'll achieve it in the next lesson.

5 Enable Fault Tolerance and Failure Detection

5.1 Handle temporary failures

Typically, distributed systems use a quorum-based approach to handle failures. A quorum is the minimum number of votes required for a distributed transaction to proceed with an operation. If a server is part of the consensus and is down, then we can't perform the required operation. It affects the availability and durability of our system.

We'll use a sloppy quorum instead of strict quorum membership. Usually , a leader manages the communication among the participants of the consensus. The participants send an acknowledgment after committing a successful write. Upon receiving these acknowledgments, the leader responds to the client. However, the drawback is that the participants are easily affected by the network outage. If the leader is temporarily down and the participants can't reach it, they declare the leader dead. Now , a new leader has to be reelected. Such frequent elections have a negative impact on performance because the system spends more time picking a leader than accomplishing any actual work.

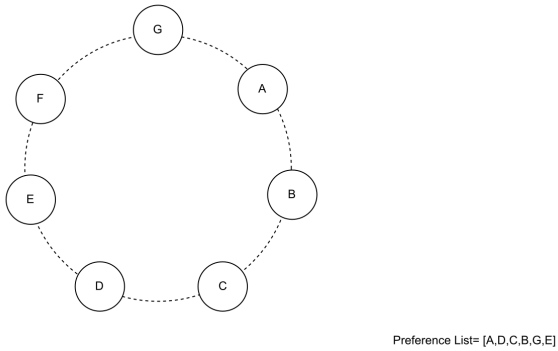
In the sloppy quorum, the first n healthy nodes from the preference list handle all read and write operations. The n healthy nodes may not always be the first n nodes discovered when moving clockwise in the consistent hash ring.

Let's consider the following configuration with n = 3. If node A is briefly unavailable or unreachable during a write operation, the request is sent to the next healthy node from the preference list, which is node D in this case. It ensures the desired availability and durability. After processing the request, the node D includes a hint as to which node was the intended receiver (in this case, A). Once node A is up and running again, node D sends the request information to A so it can update its data. Upon completion of the transfer, D removes this item from its local storage without affecting the total number of replicas in the system.

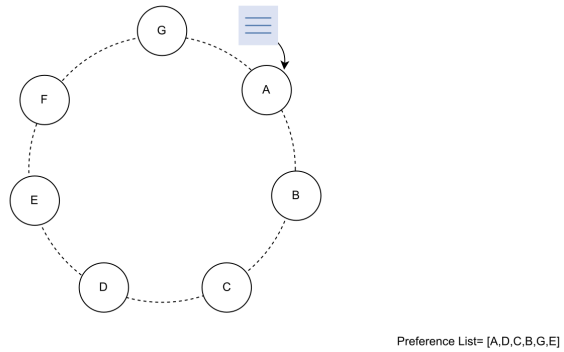
This approach is called a **hinted handoff**. Using it, we can ensure that reads and writes are fulfilled if a node faces temporary failure.

> **Note**: A highly available storage system must handle data center failure due to power outages, cooling failures, network failures, or natural disasters. For this, we should ensure replication across the data centers. So, if one data center is down, we can recover it from the other.

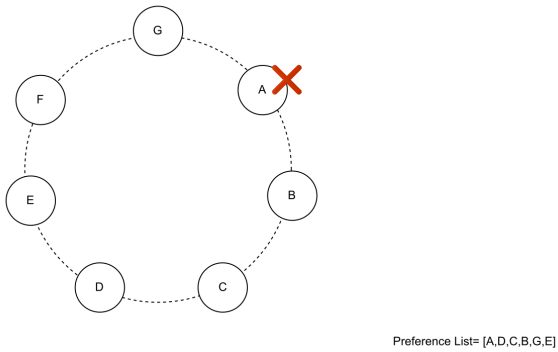
Hinted handoff helps maintain availability when nodes fail temporarily. But what happens when the system experiences prolonged failures?



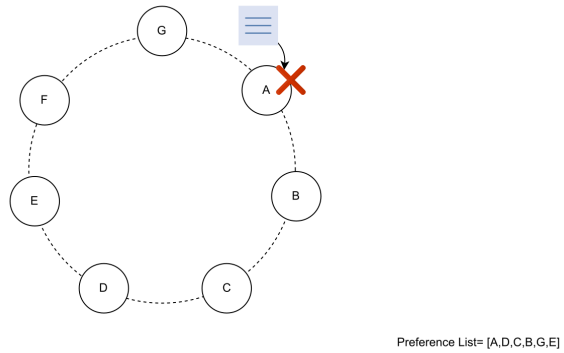
(a) Suppose we have seven nodes in our ring and a preference list of the nodes



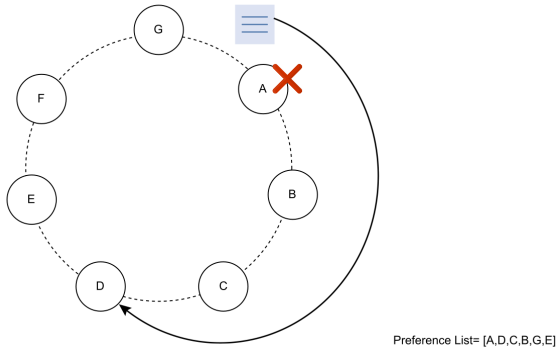
(b) A request comes in, and node A processes it since it's the next node in the ring. The node is processed while moving in the clockwise direction



(a) Node A becomes down due to some failure



(b) A request comes in. Node A has to process it but is unable to, so we check the next node in the preference list

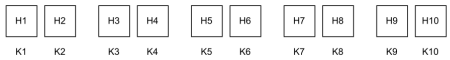


(a) Request is sent to node D as node A is down

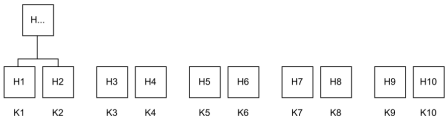
Question:
The limitations of hinted handoff
Reference Answer:
When a node remains down beyond the hinted handoff window, those temporary hints expire or are discarded to prevent unbounded storage growth. At that point, the system shifts from fast recovery to full data reconciliation. Once the node returns, background processes like anti-entropy repair, read repair, or Merkle tree comparison detect and synchronize missing updates, potentially at the cost of temporary inconsistency or data loss if recovery is delayed too long.

5.2 Handle permanent failures

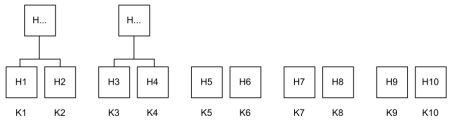
In the event of permanent failures of nodes, we should keep our replicas synchronized to make our system more durable. We need to speed up the detection of inconsistencies between replicas and reduce the quantity of transferred data. We 'll use Merkle trees for that.
In a **Merkle tree**, the values of individual keys are hashed and used as the leaves of the tree. There are hashes of their children in the parent nodes higher up the tree. Each branch of the Merkle tree can be verified independently without the need to download the complete tree or the entire dataset. While checking for inconsistencies across copies, Merkle trees reduce the amount of data that must be exchanged. There's no need for synchronization if, for example, the hash values of two trees' roots are the same and their leaf nodes are also the same. Until the process reaches the tree leaves, the hosts can identify the keys that are out of sync when the nodes exchange the hash values of children. The Merkle tree is a mechanism to implement anti-entropy, which means to keep all the data consistent. It reduces data transmission for synchronization and the number of discs accessed during the anti-entropy process.
The following slides explain how Merkle trees work:



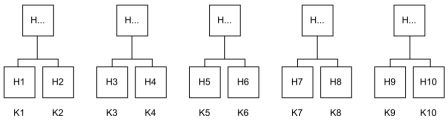
(a) Calculate the hashes for all keys. The hashes will be leaf nodes



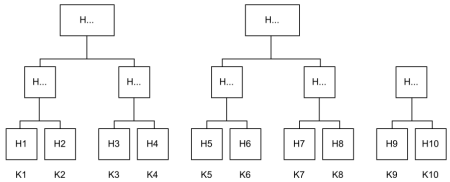
(b) The hashes of the H1 and H2 nodes are calculated and stored as their parent node



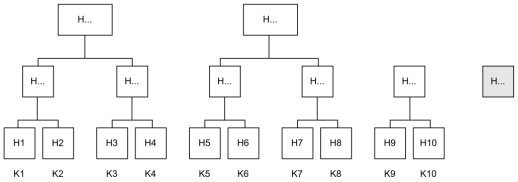
(a) The hashes of the H3 and H4 nodes are calculated and stored as their parent node



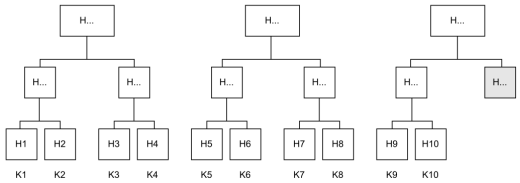
(b) The hashes of all other nodes are calculated and stored as their parent node



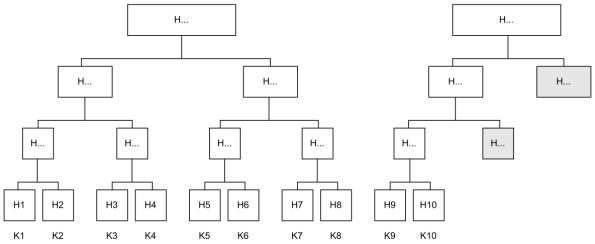
(a) The hashes of the nodes H 1 2 and H 3 4 is calculated and stored as their parent node. The same thing is done for H 5 6 and H 7 8



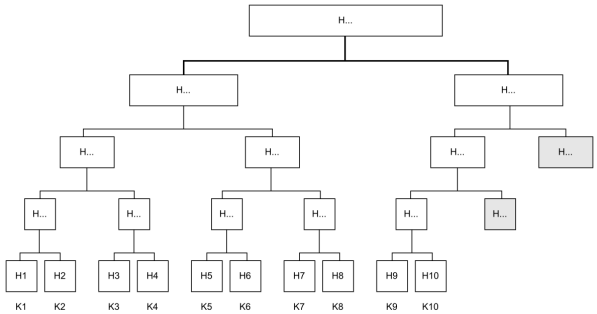
(b) We duplicate the odd node



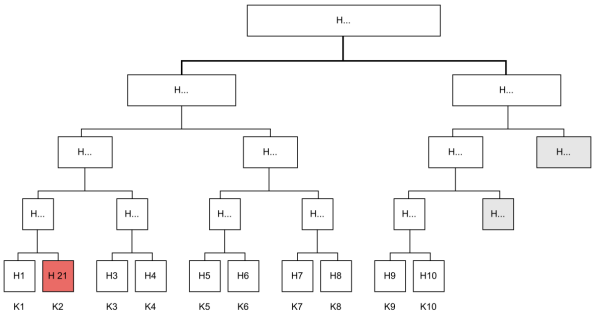
(a) Calculate the hash of both nodes (the actual node and its duplicate), and store it as the parent node



(b) Calculate the hash of the nodes and store them as the parent node

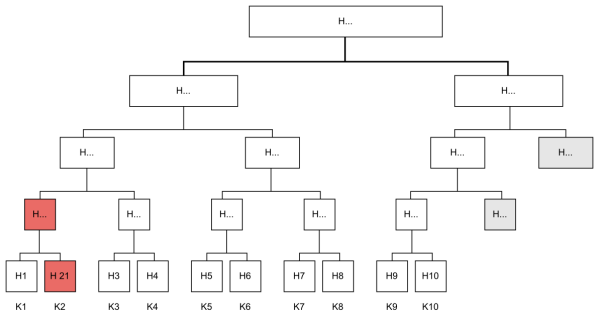


(a) Calculate the hash of the last two nodes to store as the root node

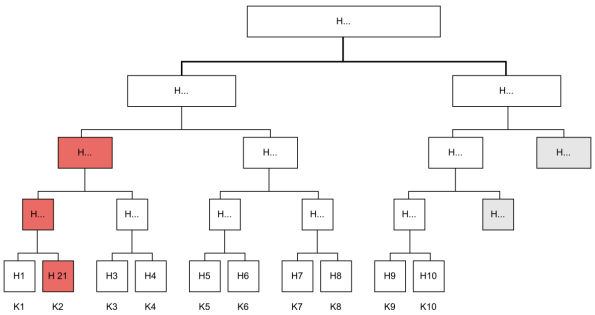


(a) Calculate the hash of the last two nodes to store as the root node

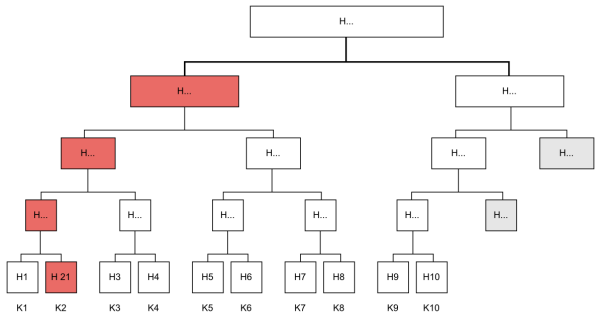
(b) Suppose the value of K2 is updated. Its hash will now be recalculated



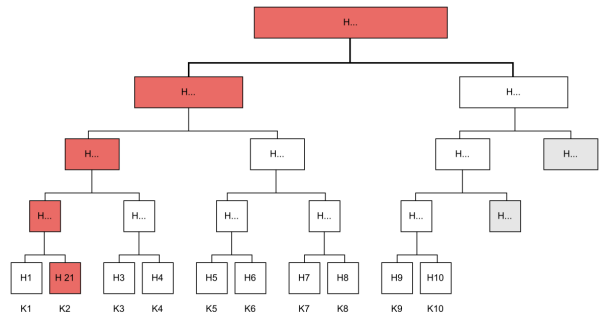
(a) The hash of the parent will also be recalculated and updated



(b) The hash of the parent will also be recalculated and updated



(a) The hash of the parent will also be recalculated and updated



(b) The hash of the root node will also be recalculated and updated

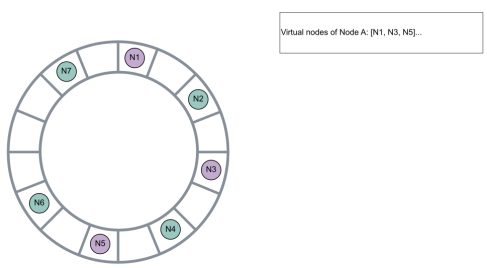
Anti-entropy with Merkle trees

Each node keeps a distinct Merkle tree for the range of keys that it hosts for each virtual node. The nodes can determine if the keys in a given range are correct. The root of the Merkle tree corresponding to the common key ranges is exchanged between two nodes. We 'll make the following comparison:

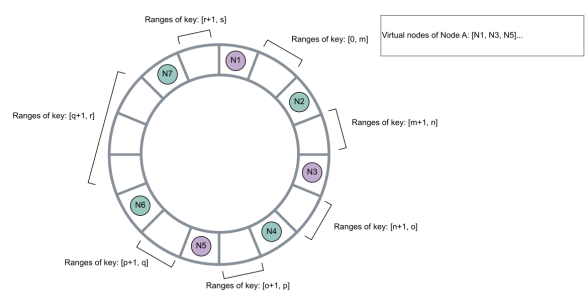
1. Compare the hashes of the root node of Merkle trees.
2. Do not proceed if they're the same.
3. Traverse left and right children using recursion. The nodes identify whether or not they have any differences and perform the necessary synchronization.

The following slides explain more about how Merkle trees work.

> **Note**: We assume the ranges defined are hypothetical for illustration purposes.



(a) Let's suppose we have the virtual nodes A and B in the ring



(b) The ranges that each virtual node covers are defined

The advantage of using Merkle trees is that each branch of the Merkle tree can be examined independently without requiring nodes to download the tree or the complete dataset. It reduces the quantity of data that must be exchanged for synchronization and the number of disc accesses that are required during the anti-entropy procedure.

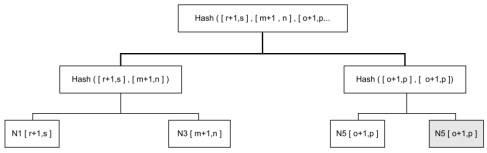
The disadvantage is that when a node joins or departs the system, the tree's hashes are recalculated because multiple key ranges are affected.

We want our nodes to detect the failure of other nodes in the ring, so let's see how we can add it to our proposed design.

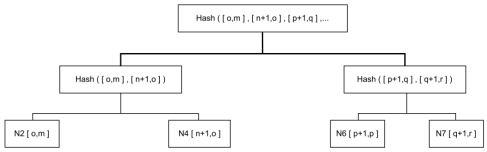
Node A	
Virtual node	Range
N1	[r+1, s]
N3	[m+1, n]
N5	[o+1, p]

Node B	
Virtual node	Range
N2	[0,m]
N4	[n+1, o]
N6	[p+1, q]
N7	[q+1, r]

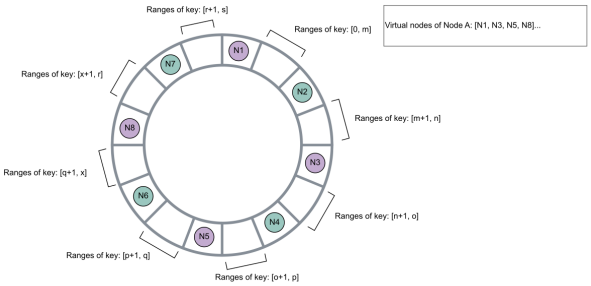
(a) The ranges that each virtual node covers are defined in table form



(b) Merkle tree for Node A



(a) Merkle tree for Node B

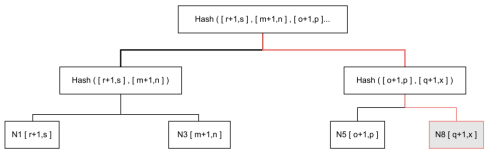


(b) Let's suppose a new virtual node, N8, of A is added and the ranges are updated accordingly

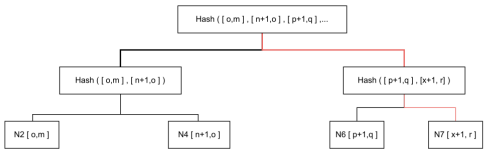
Node A	
Virtual node	Range
N1	[r+1, s]
N3	[m+1, n]
N5	[o+1, p]
N8	[q+1, x]

Node B	
Virtual node	Range
N2	[0,m]
N4	[n+1, o]
N6	[p+1, q]
N7	[x+1, r]

(a) The updated ranges that each virtual node covers are defined in table form



(b) The updated Merkle tree for Node A



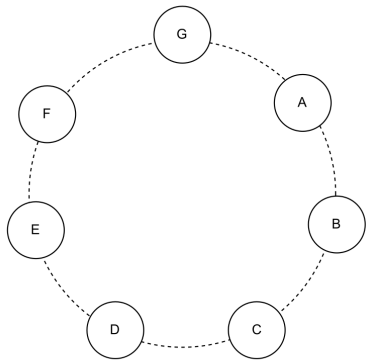
(a) The updated Merkle tree for Node B

Discuss the implications of scalability and fault tolerance when using consistent hashing in distributed key-value stores.

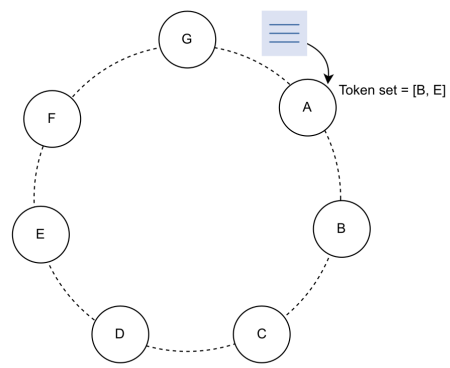
Say "Hello" to Ed in the widget below to give your answer.
Component type 'PromptAI' not yet supported.

5.3 Promote membership in the ring to detect failures

The nodes can be offline for short periods, but they may also indefinitely go offline. We shouldn't rebalance partition assignments or fix unreachable replicas when a single node goes down because it's rarely a permanent departure. Therefore, the addition and removal of nodes from the ring should be done carefully.
Planned commissioning and decommissioning of nodes results in membership changes. These changes form history. They're recorded persistently on the storage for each node and reconciled among the ring members using a gossip protocol. A **gossip-based protocol** also maintains an eventually consistent view of membership. When two nodes randomly choose one another as their peer, both nodes can efficiently synchronize their persisted membership histories.
Let's learn how a gossip-based protocol works by considering the following example. Say node A starts up for the first time, and it randomly adds nodes B and E to its token set. The token set has virtual nodes in the consistent hash space and maps nodes to their respective token sets. This information is stored locally on the disk space of the node.
Now, node A handles a request that results in a change, so it communicates this to B and E. Another node, D, has C and E in its token set. It makes a change and tells C and E. The other nodes do the same process. This way, every node eventually knows about every other node's information. It 's an efficient way to share information asynchronously, and it doesn't take up a lot of bandwidth.



(a) A set of nodes in a ring



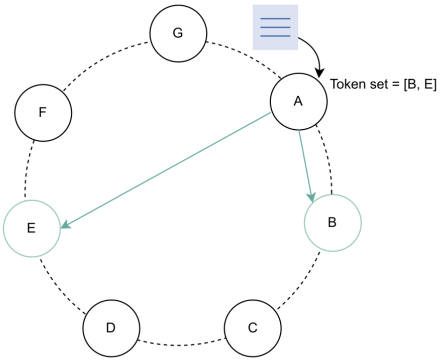
(b) Node A processes the request. Its token set has B and E in it

Test your knowledge!

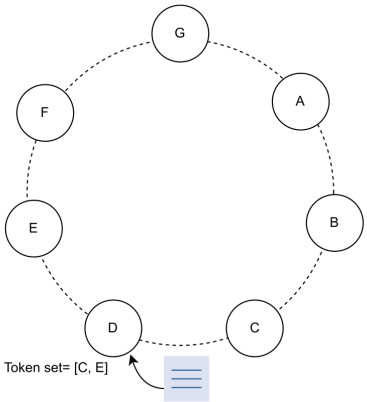
Question 1: Can the gossip-based protocol fail in a system using consistent hashing, and if so, why?

- No, the gossip-based protocol never fails as long as nodes are part of the consistent hashing ring.
- **[CORRECT]** Yes, it can fail if virtual nodes from the same physical server join the ring independently, causing logical partitioning and inconsistent updates.

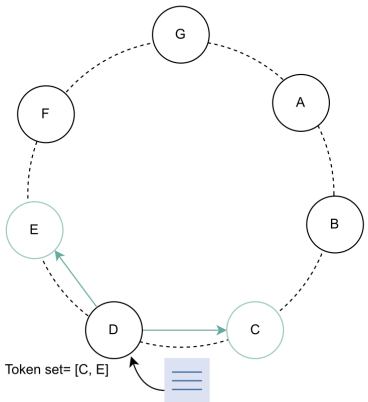
  Explanation: The gossip-based protocol can fail when virtual nodes belonging to the same physical server (e.g., $N1$ and $N2$ of node A) both believe they are independent members of the ring. This



(a) Node A gossips membership information to Node B and E after few requests



(b) Node D processes the request. Its token set has C and E in it



(a) Node D gossips membership information to Node C and E after a few requests

leads to logical partitioning, where updates keep circulating within the same node. Additionally , issues like high churn or poor node connectivity can break the gossip network, preventing updates from propagating across all nodes.

- Yes, it can fail if the hash function used in consistent hashing produces duplicate keys.
- Yes, it can fail if the system uses a single physical node for all virtual nodes.

Question 2: How can we prevent logical partitioning?

- By assigning unique hash ranges to every virtual node without overlap.
- By periodically restarting all nodes to reset their membership states.
- **[CORRECT]** By designating a few nodes as seeds through a configuration service, allowing all nodes to reconcile membership via these known seeds.

  Explanation: To prevent logical partitioning, a few nodes are configured as seeds known to every node in the system. These seeds act as reference points that help all nodes reconcile their membership information, keeping the network connected and consistent.
- By disabling virtual nodes and using only physical nodes in the ring.

Decentralized failure detection protocols use a gossip-based protocol that allows each node to learn about the addition or removal of other nodes. The join and leave methods of the arriving or leaving nodes explicitly notify the other nodes about the permanent node additions and removals. The individual nodes detect temporary node failures when they fail to communicate with another node. If a node fails to communicate to any of the nodes present in its token set for the authorized time, then it communicates to the administrators that the node is dead.

5.4 Conclusion

A key-value store provides flexibility and allows us to scale the applications that have unstructured data. Web applications can use key-value stores to store information about a user's session and preferences. When using a user key, all the data is accessible, and key-value stores are ideal for rapid reads and write operations. Key -value stores can be used to power real-time recommendations and advertising because the stores can swiftly access and present fresh recommendations.